CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 26: Dependability, Wrap-Up

Instructor: Justin Yokota

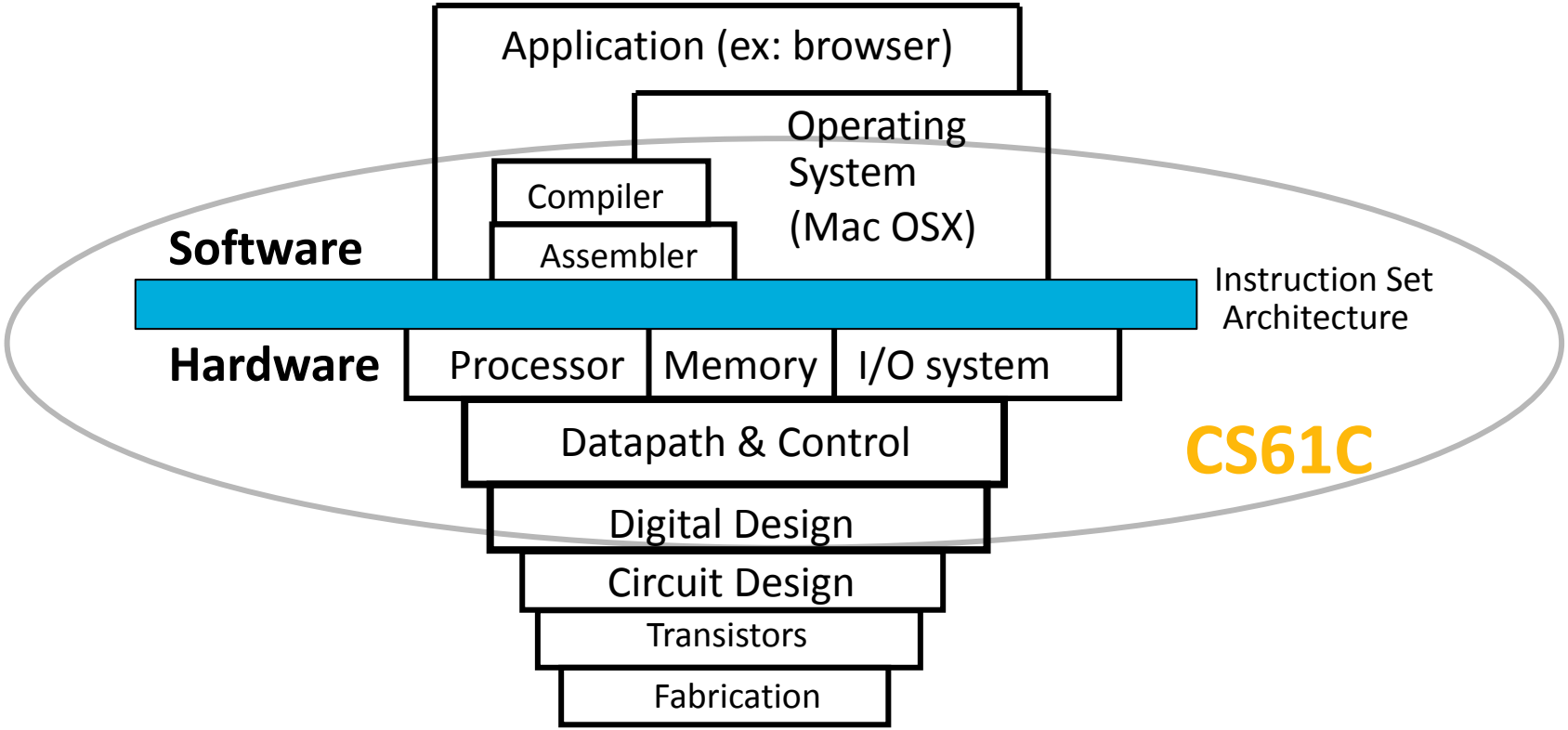
Agenda

- Class Overview
- Dependability
- Future Classes
- Q&A

Agenda

- **Class Overview**
- Dependability
- Future Classes
- Q&A

We learned Old School Machine Structures



Modern Computer Architecture is way more complicated!

Software

Parallel Requests
Assigned to computer
e.g., Search “Cats”

Parallel Threads
Assigned to core e.g., Lookup, Ads

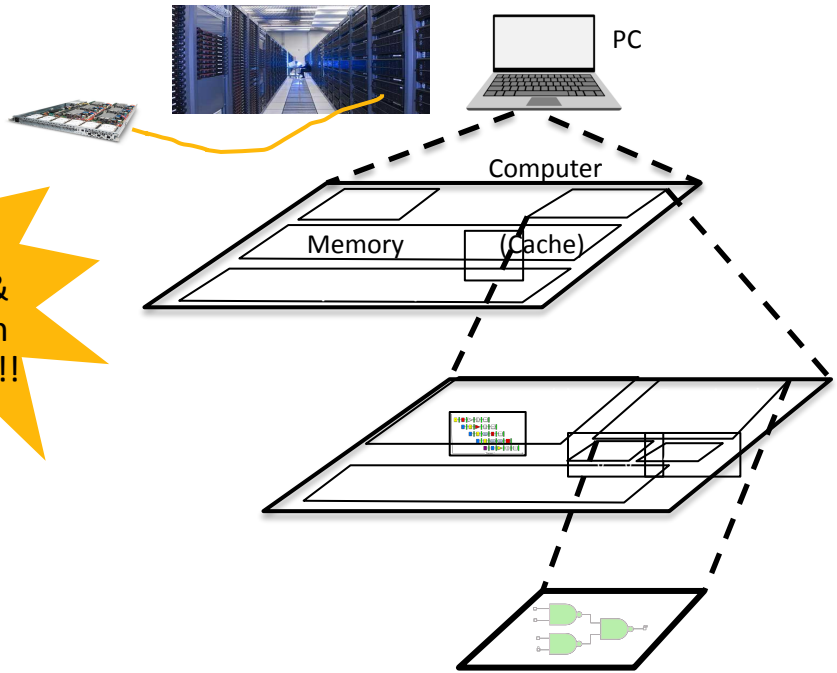
Parallel Instructions
>1 instruction @ one time
e.g., 5 pipelined instructions

Parallel Data
>1 data item @ one time
e.g., Add of 4 pairs of words

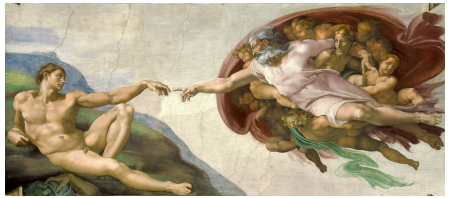
Hardware descriptions
All gates work in parallel at same time

Harness Parallelism & Achieve High Performance!!

Hardware



We made HW/SW contact!



High Level Language Program (e.g., C)

Compiler

Assembly Language Program (e.g., RISC-V)

Assembler

Machine Language Program (RISC-V)

Hardware Architecture Description (e.g., block diagrams)

Architecture Implementation

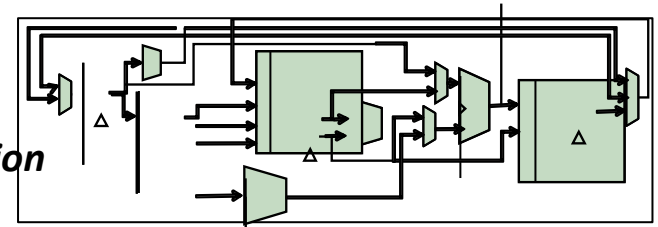
Logic Circuit Description (Circuit Schematic Diagrams)

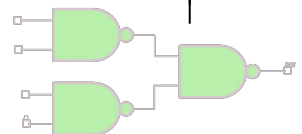
```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw    x3, 0(x10)
lw    x4, 4(x10)
sw    x4, 0(x10)
sw    x3, 4(x10)
```

```
1000 1101 1110 0010 0000 0000 0000 0000
1000 1110 0001 0000 0000 0000 0000 0100
1010 1110 0001 0010 0000 0000 0000 0000
1010 1101 1110 0010 0000 0000 0000 0100
```

Anything can be represented as a number, i.e., data or instructions





Six Great Ideas of Computer Architecture

1. Abstraction (Layers of Representation/Interpretation)
2. Moore’s Law
3. Principle of Locality/Memory Hierarchy
4. Parallelism
5. Performance Measurement & Improvement
6. Dependability via Redundancy

Agenda

- Class Overview
- **Dependability**
- Future Classes
- Q&A

Dependability

- Modern systems are fairly reliable, but errors still happen occasionally
  - Network systems: ~10^{-9} chance of bit flip
  - DRAM: Cosmic rays at Earth sea level give a ~10^{-13} chance of a specific bit flipping per second (2009).
  - Error rates are higher for more modern systems (due to there being lower difference between a 1 bit and a 0 bit), and generally higher in space/high altitude environments.
  - Larger failures could damage the entire disk, with probability ~1% per year.
- Seems like a small amount, but we often deal with GiB at a time.
  - Downloading a 2 hour movie would have ~2 GiB * 10^{-9} = 10-20 bit errors in the download
- Conclusion: Failure is not an option. It is mandatory.
- We need a way to mitigate the effects of failure.

Availability: Definitions

- Motivating example: You have some tool/service. Every now and then it fails, so it's unavailable until you fix it.
- Mean Time to Failure (MTTF): Average time it takes for an entirely new system to fail.
- Mean Time to Repair (MTTR): Average time it takes to fix the tool once it breaks.
- Mean Time Between Failures (MTBF): Average time to complete one "cycle" of working->broken->working.
  - Equal to MTTF+MTTR
- Availability: Percentage of the time that the service is available
  - Equal to MTTF/MTBF

Availability

- Common shorthand is "9s of reliability/availability"
  - 1 nine of availability -> 90% available
  - 2 nines of availability -> 99% available
  - 3 nines -> 99.9%
  - 5 nines of availability ->99.999% availability = 5 minutes of repair/year
    - Extremely expensive to maintain
- Another common metric is Annualized Failure Rate: average number of failures per year.
  - Often around 1-10%, for individual drives, depending on the age of the drive.

Availability

- What happens if we have multiple components?
- If all our components are critical, then we expect the overall system to fail *faster* than any individual component
  - If your battery breaks every two years and your CPU breaks every two years, one of them will break (on average) every year.
- On the other hand, if we have two copies of the same system and only need one of them to work, it's extremely unlikely to have both systems fail simultaneously
  - If two different batteries have an availability of 99%, at least one battery will be working 99.99% of the time.
    - On the other hand, you'll be at half efficiency 2% of the time.
- Major principle in engineering: Add redundancy to systems to reduce the chance of failure
  - Eliminate all single points of failure.

Dependability Through Redundancy

- This ends up showing up a lot in various engineering components
  - From a hardware perspective, laptops are designed to "fail slowly" by shunting work to any remaining working components
    - This is why laptops tend to get slower as they age
- Today, we'll discuss two specific cases of data redundancy:
- Error Correcting Codes
  - Recover correct binary data even if some bits were corrupted
- RAID
  - Connect multiple hard drives together so that even if one fails, all data is recoverable

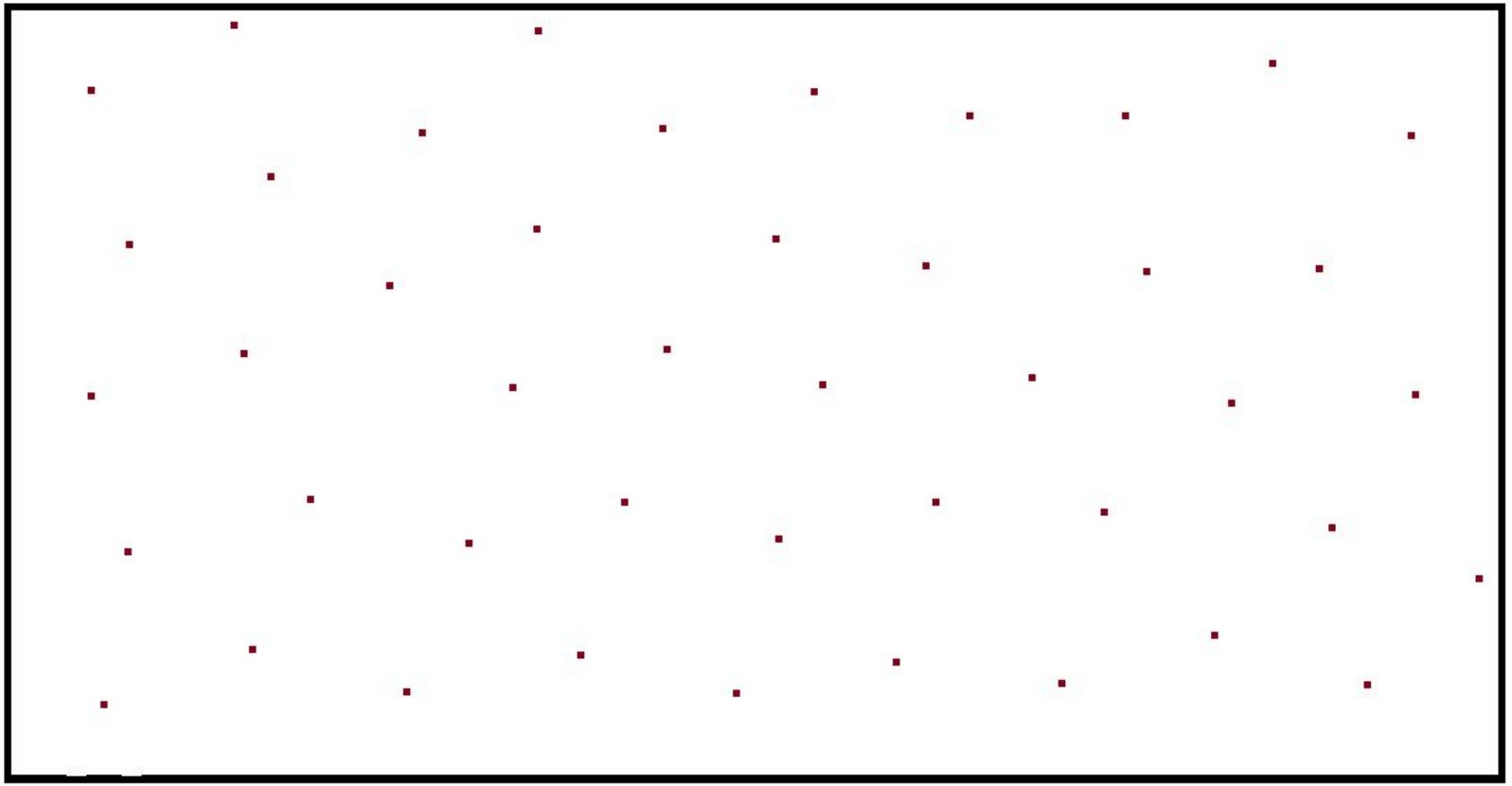
Error Correcting Codes

- When storing data, there's a very low (but still possible) chance that some of our bits get flipped by cosmic rays.
- In order to detect these, we need to store more bits than the initial data we received.
- Two major goals:
  - Detect if an error occurred
    - Even if we can't fix the error, we can try again/inform the user that something wrong happened.
  - Correct an error that happened
    - Generally harder than just detecting an error, but lets you continue running the program even if an error happens.
- General assumption is that there's either a fixed number of errors (since it's rare to have multiple errors at once), or a fixed percentage of errors.
  - Ex. A Hamming Code might be able to detect 2-bit errors, and correct 1-bit errors.
  - Ex. QR codes use a Reed-Solomon code that can correct up to 7-30% errors, depending on the correction rate requested.

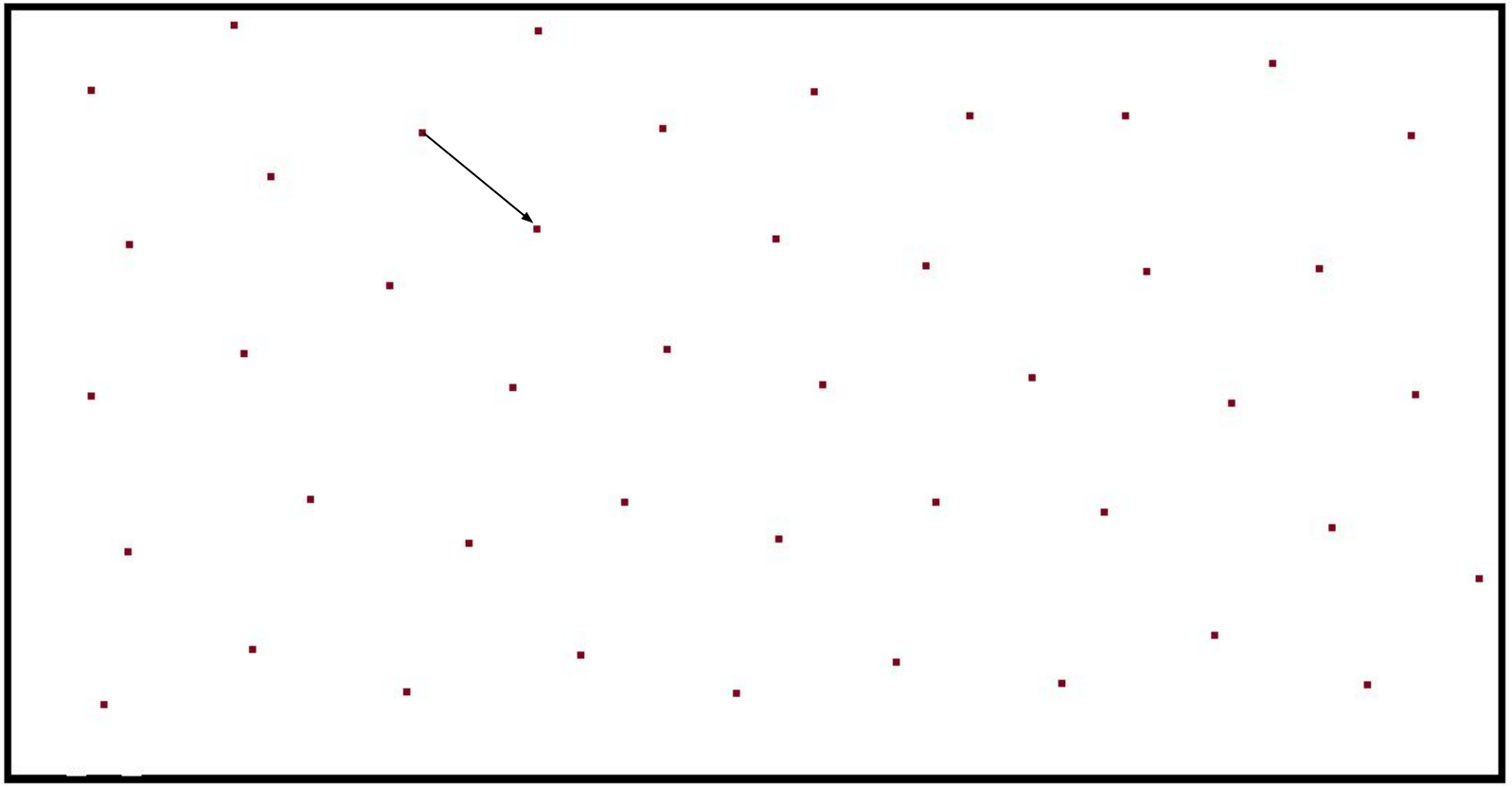
Error Correcting Codes

- Example: We start with 4 data bits (16 possible data values)
- We use some function f to convert from 4 data bits to a 7-bit codeword
  - The 16 possible outputs of f are considered the "valid" codewords, because they signify a lack of corruption.
- Someone takes that codeword and flips up to 1 bit.
  - Defensive programming: Assume that the bit flip is malicious and actively trying to make your system fail.
- If we are guaranteed that a corruption doesn't get us to another valid codeword, we can detect 1-bit errors
- We receive those 7 bits (with potentially one corruption) and use some function g to convert back to 4 data bits.
- If we are guaranteed to get back our old data, we can correct 1-bit errors

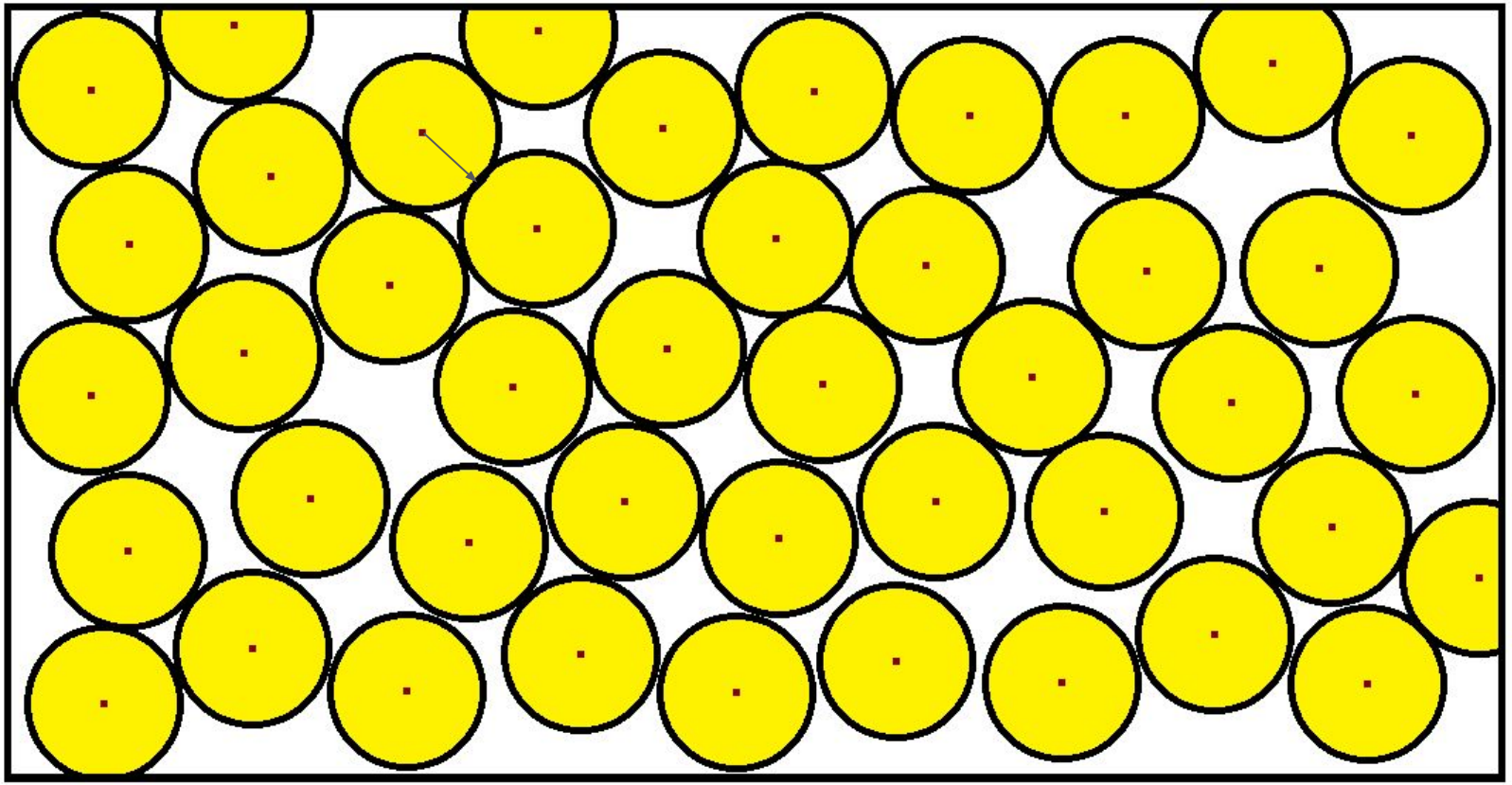
Error Correcting Codes: Graphic



Error Correcting Codes: Maximum to Detect Error

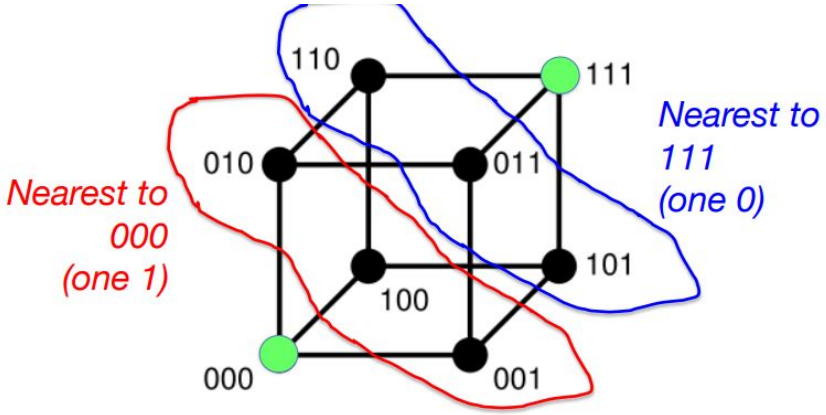


Error Correcting Codes: Maximum to Correct Error



Error Correcting Codes

- Generally, you can detect about twice as many errors as you can correct.
- As long as your bit space is large enough, you can create an ECC. The goal is to try to find the smallest possible "box" (bit count) that lets us fulfill a given detection/correction requirement.
- Reality is more discrete, and n-dimensional, but otherwise the same as the 2-D version (the same concepts work as long as we use a metric space)
    - If you can correct n-bit errors, you can generally detect up to 2n errors (diagram)
- Hamming Distance: Number of 1-bit corruptions needed to move from one bitstring to another
- Limited by the two closest codewords



Parity Bit

- Attempt 1: Store one extra bit so that the parity (number of 1 bits) of the string is even
  - Ex. If we want to store data 0b1001 1000, our parity bit will be 1, so we store 0b 1 0011 0001
  - Ex. If we want to store data 0b1001 1001, our parity bit will be 0, so we store 0b 1 0011 0010
- To convert back, we can just cut the parity bit.
- Can we detect 1-bit errors?
  - Yes. If one bit gets corrupted, our parity will become odd instead
- Can we detect 2-bit errors/correct 1-bit errors?
  - No: The two example codewords are Hamming distance 2 apart, so if we had two corruptions, we could go from one codeword to another.
- Is this optimal to detect 1-bit errors?
  - Yes. We can't do better than "1 extra bit", since "0 extra bits" would have no invalid codewords.

Hamming Code

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15	
Parity bit coverage	p1	✓		✓		✓		✓		✓		✓		✓		✓		✓		✓		
	p2		✓	✓			✓	✓			✓	✓			✓	✓			✓	✓		
	p4				✓	✓	✓	✓					✓	✓	✓	✓					✓	
	p8								✓	✓	✓	✓	✓	✓	✓	✓						
	p16																✓	✓	✓	✓	✓	

- Main idea of Hamming Codes: Include multiple parity bits (that look at a subset of all bits), so we can pinpoint which bit got corrupted
- In the above, d1, d2, … refer to the first, second, … data bits, and p1, p2, … are the parity bits.
- Parity bits are chosen such that the bits checked on each row have even parity
- Note that no row has multiple parity bits checked, so each parity bit is uniquely determined

Hamming Code: Encoding

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15
Parity bit coverage	p1	✓		✓		✓		✓		✓		✓		✓		✓		✓		✓	
	p2		✓	✓			✓	✓			✓	✓			✓	✓			✓	✓	
	p4				✓	✓	✓	✓					✓	✓	✓	✓					✓
	p8								✓	✓	✓	✓	✓	✓	✓	✓					
	p16																✓	✓	✓	✓	✓

- Example: Encode 0b 011 0111 0110 as a Hamming code
- __0_110_1110110
- _ _ 0 _ 1 1 0 _ 1 1 1 0 1 1 0 -> Even parity, so p1=0
- 0 _ 0 _ 1 1 0 _ 1 1 1 0 1 1 0 -> Even parity, so p2=0
- 0 0 0 _ 1 1 0 _ 1 1 1 0 1 1 0 -> Even parity, so p4=0
- 0 0 0 0 1 1 0 _ 1 1 1 0 1 1 0 -> Odd parity, so p8=1
- 000011011110110

Hamming Code

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15	
Parity bit coverage	p1	✓		✓		✓		✓		✓		✓		✓		✓		✓		✓		
	p2		✓	✓			✓	✓			✓	✓			✓	✓			✓	✓		
	p4				✓	✓	✓	✓					✓	✓	✓	✓					✓	
	p8								✓	✓	✓	✓	✓	✓	✓	✓						
	p16																✓	✓	✓	✓	✓	

- To decode a Hamming Code:
- Check the parity of each row. If all the rows are correct, no error occurred
- If one bit gets corrupted, that'll flip the parities on its column.
- Note: Each bit (including parity bits) has a unique set of parity bits it affects, so we can always identify which bit got corrupted
- Using that information, we can figure out fix the corrupted bit, and return the correct data
- Useful note: If we label bits from 1-n (left to right), the corrupted bit is exactly the binary representation of our parities

Hamming Code: Encoding

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15	
Parity bit coverage	p1	✓		✓		✓		✓		✓		✓		✓		✓		✓		✓		
	p2		✓	✓			✓	✓			✓	✓			✓	✓			✓	✓		
	p4				✓	✓	✓	✓					✓	✓	✓	✓					✓	
	p8								✓	✓	✓	✓	✓	✓	✓	✓						
	p16																✓	✓	✓	✓	✓	

- Example: Decode 0b 000 0110 1110 0110 as a Hamming code
- 000011011100110-> Odd parity, so 1
- 000011011100110-> Odd parity, so 1
- 000011011100110-> Even parity, so 0
- 000011011100110-> Odd parity, so 1
- Overall parity is 0b1011 = 11, so bit 11 is corrupted
- 000011011100110
- 0 110 1100110 (Remove parity bits)

Hamming Code

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15	
Parity bit coverage	p1	✓		✓		✓		✓		✓		✓		✓		✓		✓		✓		
	p2		✓	✓			✓	✓			✓	✓			✓	✓			✓	✓		
	p4				✓	✓	✓	✓					✓	✓	✓	✓					✓	
	p8								✓	✓	✓	✓	✓	✓	✓	✓						
	p16																✓	✓	✓	✓	✓	

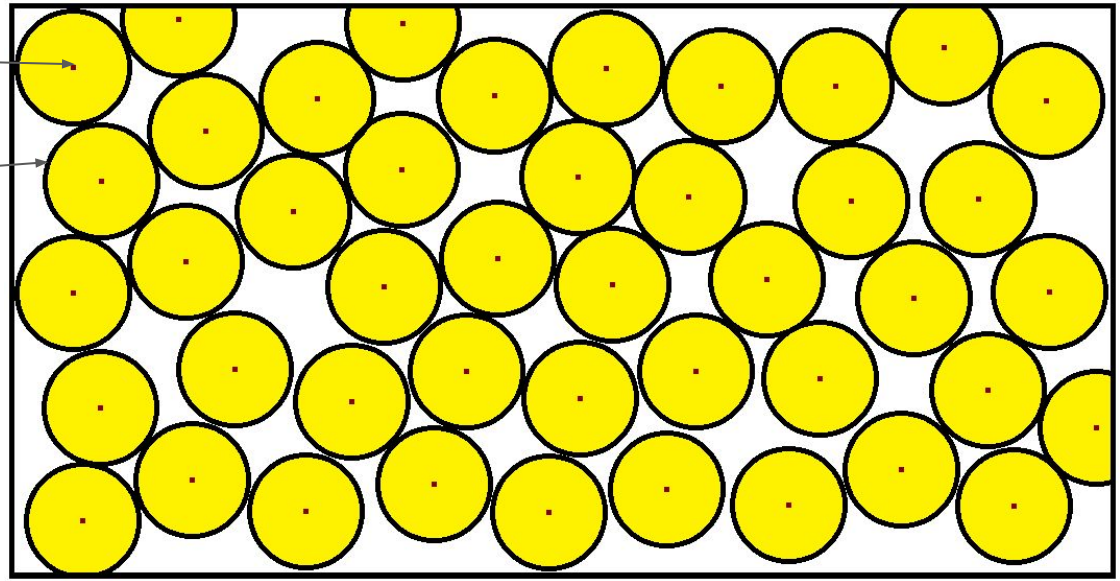
- Thus, allows for correcting 1-bit errors
- Can we detect 2-bit errors?
  - Yes, since no two bits flip all parities an even number of times
- Can we detect 3-bit errors?
  - No; if bits 1,2,3 were corrupted, all our parities would be even.
- Is this optimal to correct 1-bit errors?
  - Yes, because the total "area" covered by yellow circles almost equals the box size
  - Note: CS 70 describes an ECC that can correct 1 error with 2 extra packets (Berlekamp-Welch). This doesn't apply here because Berlekamp-Welch allows only up to n packets sent at once, where n is the number of letters in our alphabet. For binary, Berlekamp-Welch would only be able to send at most 2 bits at a time.

Hamming Code Optimality

Every possible codeword is either
1) a correct codeword (red dot), or
2) corrects to a correct codeword (in circle)

Assuming the total number of bits is $2^n - 1$, this is optimally packed

It is close enough to optimality that you can't do better by removing a parity bit for bit counts that are not $2^n - 1$



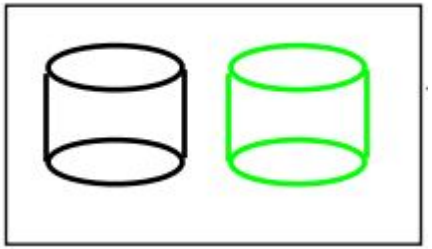
RAID

- Generally, the larger you make a disk, the faster it breaks. As such, it's useful to set up a system where you use many disks together
  - Goal: Ensure that even if some disks fail, you still have all your data
  - Goal: Maximize the amount of data you can store
- RAID: Redundant Array of Independent Disks
- 7 different formats (called levels) that can be used to save data (RAID 0 to RAID 6)
- RAIDs 2, 3, and 4 are mostly obsolete nowadays, but we'll still mention them (since they lead into RAID 5)
- Note: The data written on the disks get Hamming-encoded, so single-bit errors aren't a concern. If a disk fails, we'll know which disk failed, and will lose all data on it.
- We'll use for most of these examples a system where we have 4 1 TiB disks we want to put into a RAID array.

RAID 0

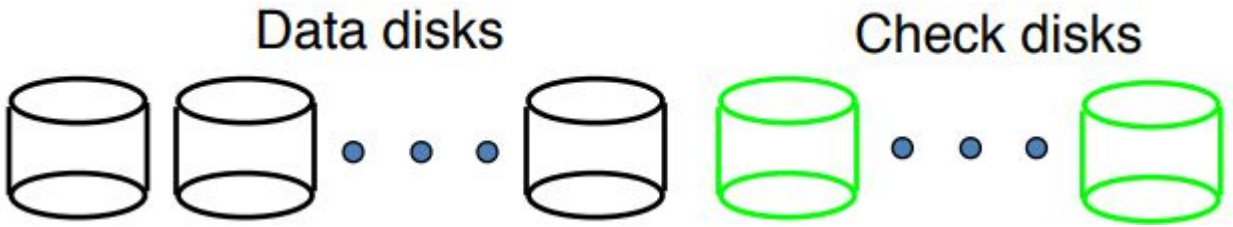
- No redundancy
- Split data over all disks
- Ex. 4 1TiB drives combine to store 4 TiB total
- Likely better than a single 4 TiB drive because each disk can output data independently (so 4x bandwidth)
- Generally useful if redundancy isn't needed
- On the other hand, if any disk fails, the entire array fails (because data was lost)

RAID 1



- Maximum redundancy
- Each disk stores an exact copy of the data
- Ex. 4 1 TiB drives combine to store 1 TiB, and three backups
- Very reliable (almost always available), but also extremely expensive
- Useful for when a system cannot be allowed to fail (ex. life support systems), or when memory use isn't too important.

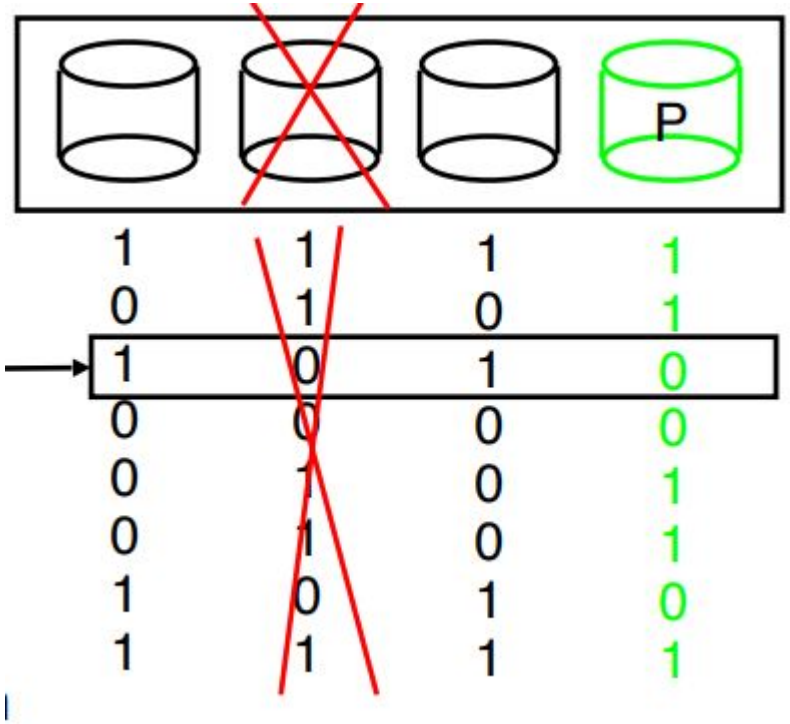
RAID 2



- Split the data into several data disks
- Each bit gets encoded with a Hamming Code, split across all disks
- Ex. If we had 7 1 TiB disks, we would save 4 TiB worth of data. The remaining 3 disks will be used to store parity bits such that the first bit of each disk form a valid 7-bit Hamming code, and so on.
- Hamming Code guarantees that if at most 2 disks of the 7 fail, you can recover
- Basically never used; if you want to recover from two disk failures, use RAID 6 :D

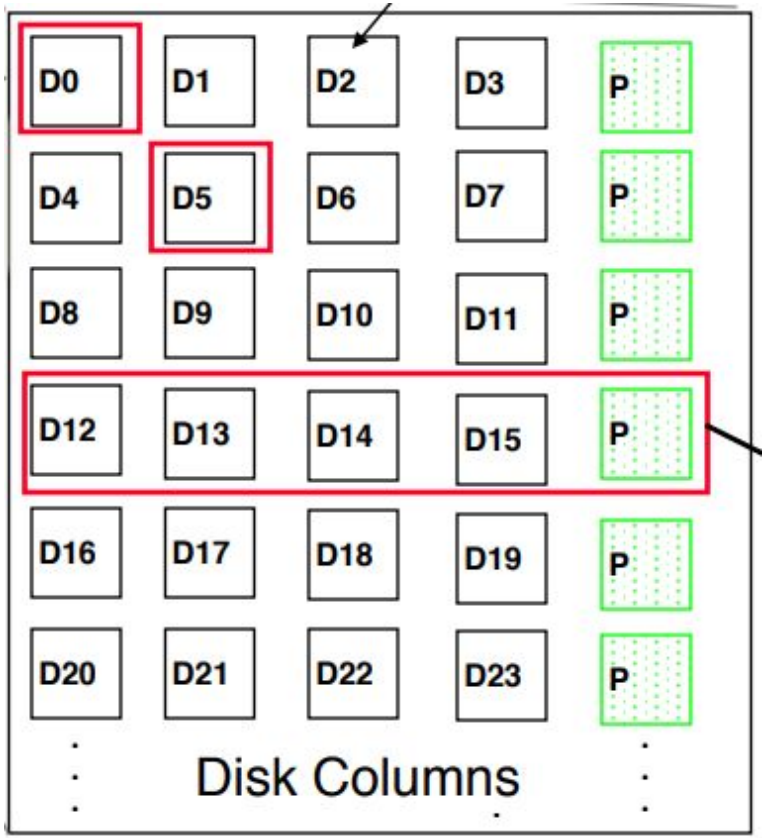
RAID 3

- Split the data into several data disks
- The last disk is a parity disk, and stores the parity bits of the remaining disks
- Ex. 4 1 TiB disks will have 3 TiB of storage and 1 parity disk
- If a disk fails, we can recover the old data by taking the parity of all remaining disks. Thus, we can recover from one disk failure.
- Never used, since worse than RAID 4



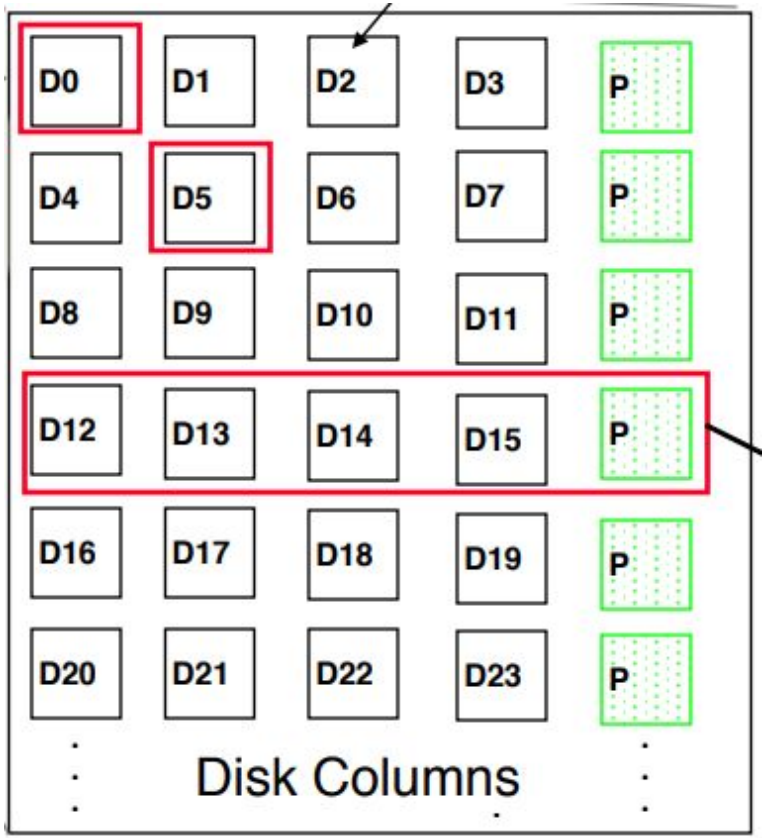
RAID 4

- Same as RAID 3, but we instead work at a block level
- Instead of writing one byte at a time, chunk memory into blocks of ~100 KiB, and read/write blocks as needed
- Use caches to combine multiple small reads/writes in a single operation



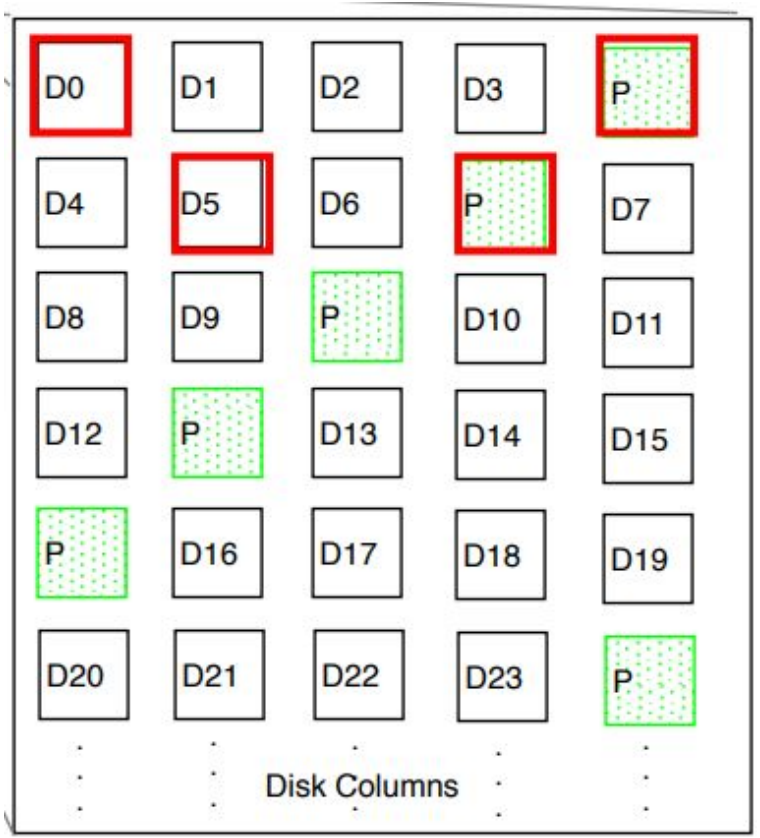
Problem with RAID 4

- Generally, it is possible to do one operation per disk (A single disk can do a single read/write at a time)
- It's also possible to do reads/writes from two different disks at the same time.
- Ex. In the image on the left (5 disks), if we want to read D0 and D5, we can do so at the same time (1 unit of time). If we wanted to read D0 and D4, though, it would take 2 units of time.
- For 100 memory accesses, parity disk is accessed all 100 times = bottleneck.
- Memory accesses are way longer than math operations, so we only really care about disk reads/writes



RAID 5

- Same as RAID 4, but this time, we interleave parity blocks over all disks
- If we want to write to both D0 and D5, disks 1, 2, 4, and 5 all get one write each, so we can finish all writes in 1 unit of time
- If we want to write to 100 random blocks, each disk will get ~40 accesses each, so this evens up the work



RAID 6

- RAID 5's good if we only want to handle one disk failure at a time
  - As long as disk failures are independent, RAID 5 will likely work
- However, failures often happen in close proximity
  - If the Golden Gate Bridge collapses due to earthquake, there's a good chance the Hayward Bridge will also collapse due to earthquake fairly soon…
- RAID 6 introduces a second parity block, (so we get one fewer disk to store data).
- This allows us to handle up to two disk failures
  - Since we work with blocks instead of bits, the number of disks is less than the number of letters in our alphabet, so we don't need to do Hamming Encoding, and can just do Berlekamp-Welch or a derivative

Other RAID options

- We can nest RAID levels to fine-tune our array to the situation
  - Ex. If we have 4 disks, we can treat it as two sets of 2 disks each, with the disks in a set having identical data. This is called RAID 10 (“one zero”), since it's two RAID 1 systems that combine in a RAID 0 format.
- We can also keep a spare disk entirely unused (but already installed), so we can replace it quickly when one disk fails, to reduce the chance that a third disk fails before we finish repairing the first one.
  - Called a "hot spare".

Warehouse-Scale Computers

- Massive datacenters: 10,000 to 100,000 servers + networks to connect them together
  - Emphasize cost-efficiency
  - Attention to power: distribution and cooling
- (Relatively) homogeneous hardware/software
- Offer very large applications (Internet services): search, social networking, video sharing
- Very highly available: < 1 hour downtime/year
  - Must cope with frequent server failures *without* killing the entire datacenter

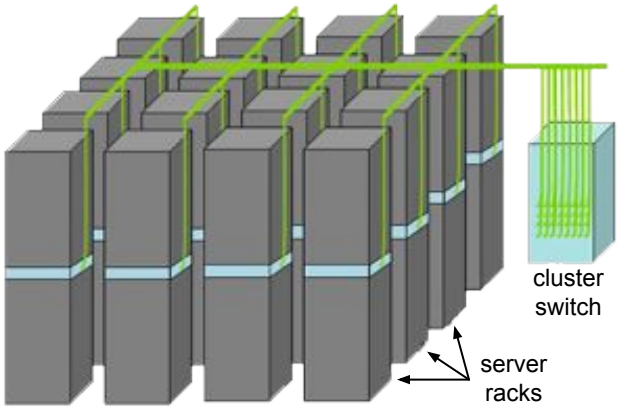
E.g., Google's Oregon WSC

Containers in WSCs

Inside WSC

Inside Container

Equipment Inside a WSC



Server (in rack format):
1 ¾ inches high “1U”,
x 19 inches x 16-20 inches: 8 cores, 16 GB DRAM, 4x1 TB disk

7 foot Rack: 40-80 servers + Ethernet local area network (1-10 Gbps) switch in middle (“rack switch”)

Array (aka cluster):
16-32 server racks + larger local area network switch (“array switch”) 10X faster
 cost 100X: cost f(N^2)

Server, Rack, Array

Design Goals of a WSC

Many goals unique to Warehouse-scale Computers.

- Ample parallelism:
  - Batch apps: large number independent data sets with independent processing.
- Scale and its Opportunities/Problems
  - Some things get more expensive in bulk (ex. failure rates)
    - Must prepare for high failure rates
  - But some things get cheaper when you buy in bulk
- Operational Cost:
  - Goal: ensure that operational costs << price to rent out server time

Main Engineering Hurdles of Warehouse-Scale Computers

- Electricity use
  - Many computers = high electricity use
  - Peak loads often significantly larger than off-peak loads
    - Ideally want power use to scale linearly with load
- Heat Dissipation
  - Many computers in the same room produce significant heat
  - Computers need cool temperatures ideally to work
  - Represents a significant fraction of energy cost that cannot be reduced during off-peak hours
- Server Failures
  - Servers need to be replaced constantly; if you have 100000 servers and a 5% annual failure rate, 5 servers fail per day
  - When working with massive programs, have to handle the fact that one of the servers you are using will likely crash during your runtime.
  - In practice, systems like RAID work, but at a much more complicated scale

August 2026 AWS Instances & Prices

Instance	Per Hour	$ Ratio to Small	Virtual Cores (vCPU)	Memory (GiB)	Disk (GiB)
Standard Small (t3.small)	$0.017	1	2	2	EBS
Standard Large (t3.large)	$0.067	4	2	8	EBS
Standard 2x Extra Large (t3.2xlarge)	$0.269	16	8	32	EBS

At these low rates, Amazon EC2 makes money! (even utilized 50% time)

EBS = Elastic Block Store (SSD=$0.08/GB-mo, HDD=$0.045/GB-mo)

Other options with dedicated attached SSD if you choose & pay for that

Agenda

- Class Overview
- Dependability
- **Future Classes**
- Q&A

What next?

- Kind of depends on what you plan to do in CS. After 61C, the major expands significantly
- Hardware track
  - Focus on designing new/more efficient/cheaper systems/components
  - Extend current technologies to new devices, assembly languages, etc.
  - Ex. Chip design, Manufacturing
- Software track
  - Develop software for new applications
  - Alternatively, improve on current algorithms/compilers
  - Includes most cross-major interests
  - Ex. App development, security, data analysis
- Theory track
  - Expand current programming paradigms
  - Ex. Nondeterministic programming, complexity classes
- Combined paths/Other tracks?
  - Ex. CS Education
- Regardless of what track you choose, it's useful to be somewhat-versed in the other tracks.

Hardware

- Datapath
  - If you enjoyed Project 3, this is a great path to look into
- Caches and the Memory Model
- VM and Assembly
  - Many aspects of these are defined by what makes the hardware go faster
- Parallelism
  - Nowadays, we're transitioning to warehouse-scale computing with thousands of cores
- In the standard CS pipeline, 61C is the first course that really goes into the hardware side of things
- I'm not too familiar with the hardware side…

Software

- If 61A and 61B were "How to program", 61C is "How to program efficiently"
- C programming
  - Still fairly popular, despite its difficulty of use and horrible security
- Assembly, CALL, Pipelining
  - For compiler design
  - Lets you write remarkably fast and horribly unreadable code
- Parallelism
  - The problems we want to solve keep getting bigger, and the only way to solve bigger problems is to use parallel programming
- OS, IO, Caching
  - Knowing how things work at the lower level helps make design decisions

Theory

- Not much directly in 61C, but there's a few aspects
- Number Rep/Floating Point
  - Mostly this relates to the aspect of assigning binary data meaning. Assigning meaning in a manner that is useful is rather tricky.
- CS theory is fairly similar to applied math and proofs (see CS 170)

Connections

- It's generally useful to know a bit of all tracks
- Hardware and software influence each other; you write software that's fast on a given hardware, and make hardware that allows more software to be fast
- Software applications normally dictate what theoretical projects get funding, and writing software often starts with knowing what's theoretically possible

CS 170

- Algorithm Design
- Primary focus: Software, Theory
- Given a problem, what asymptotic runtime can you achieve to solve it?
- Often useful to see how to optimize asymptotic runtime first; if you can get an n^2 algorithm down to n log n, that'll almost always be better than a parallel algorithm
- One of the most important software-track classes to take, and one of the few theory-track classes available
    - Commonly considered the class that helps solve interview problems, so useful to take early

CS 161

- Computer Security
- Primary focus: Hardware, Software, Theory
- How to hack programs, and how to prevent your programs from being hacked
- How the internet works (because the internet is terribly insecure)
- Ends up affecting all layers of the tech stack, so it's fairly fast-paced
- Very important to take, but it's also not urgent

CS 162

- Operating Systems
- Primary focus: Hardware, Software
- Expands on the concepts of the OS and VM
- Uses C a lot, so it helps to be comfortable with explicit memory management
- There's also a lot of multiprocess programming
  - Locks, deadlocks, timers, etc.
- The main project sequence goes through writing a simple OS, and adding various bits of functionality to it.

CS 164

- Compilers
- Primary focus: Software, Theory
- The "C" part of CALL
- Has a lot of connections to linguistics and parsing
- The main project sequence goes through writing a compiler from (a variant of) Python to RISC-V, step by step
  - Lexer: Translate code into sequences of tokens (letters->words)
  - Parser: Translate tokens to meaningful statements (words->sentences)
  - Code Generation: Translate meaningful statements into the new language (sentences->new language)

CS 188/189

- AI and Neural Nets
- Primary focus: Software
- Mostly linear algebra, but it's a fairly big topic nowadays

CS 172

- Computability and Complexity
- Primary focus: Theory
- One of the most theory-heavy classes for undergraduates
- Expands on FSMs and other theoretical computation models
- Computability is all about what can be computed in a given model (ex. showing that some programming language is Turing-Complete)
- Complexity is about the difficulty of solving problems (ex. P, NP, PSPACE, LOG, coNP)

EECS 151, CS 152

- Introduction to Digital Design, Computer Architecture
- Primary focus: Hardware
- Expands on Project 3 and the circuitry aspects of this course
    - Also expands on parallelism and performance aspects!

CS 152

- Branch prediction, remember her?
  - It's an entire research field, and there are many different methods by which we can improve execution!
  - Bit-wise manipulation + tracking, mostly
- One-instruction executed per cycle max (even during pipelining)
  - No longer the case with instruction-level parallelism!
  - ILP: multiple instructions get executed per clock cycle!
  - Architecture that supports ILP include
    - Superscalar architecture
    - VLIW: very long instruction words (152)
- Vectorised architecture, SIMD which is Intel-supported
  - What about RISC-V vectorised architecture?
  - Currently undergoing research at Berkeley!
  - BAR: Berkeley Architecture Research
  - (Vectorising radix-based sorting algorithms?)

EECS 151

- Lab-based class: FPGA or ASIC (or both!)
  - FPGA: Field Programmable Gate Array, will be hands-on based (working with an FPGA board)
  - ASIC: Application-Specific Integrated Circuit, will be simulated
  - Both different ways in building and designing chips to be shipped for production!
- Focuses on datapath/SDS sections of course
  - A lot of things we've abstracted away in 61C become clearer at this level!
  - Delays of hardware components
  - Various gates and how they're created
  - FSMs
  - More datapath
- Introduces Verilog

EE 149, 143

- Introduction to Embedded Systems, Microfabrication Technology
- EE 149
  - Embedded system: special purpose system (CPU, memory, I/O) designed to perform singular or few specific functions
  - Basic analysis, control, and systems simulation
  - How systems interface with real world usage
  - Embedded platforms ⇒ OS things
  - Distributed embedded systems
- EE 143
  - Microfabrication: how all of this stuff on chips/hardware gets actuated (typically on the micro-scale and on silicon chips)
  - " Thermal oxidation, ion implantation, impurity diffusion, film deposition, expitaxy, lithography, etching, contacts and interconnections, and process integration issues"
  - Transistors!

200-level classes

- Many graduate-level classes also allow undergraduates to enroll (after graduate enrollment)
- Often go deeper into concepts covered in upper-division classes
  - CS 267: Parallel computing
  - CS 263: Compilers
  - CS 294: Various topics (based on the semester)
- Many of these classes are rarely taught, so take them when you can!
- Primary difference from undergraduate classes: Most graduate courses have no final, but instead have a final project, where you design or analyze an emerging research topic.

Other classes

- A lot of math classes synergize really well with CS (Disclaimer: I was a math major)
  - Math 110: Linear Algebra, useful for AI and EE
    - One of the few fields of math that is considered "mostly solved", so if you can express a problem as linear algebra, you can solve it.
  - Math 128A: Computation, useful for engineering in general
    - Expands on the error rates you get from floating point computation and real-world data, and how that affects future computations. Unfortunately, it uses MATLAB.
  - Math 113: Abstract Algebra, useful for theory
    - Groups, rings, and fields. Ends up similar to how objects work. Lots of encoding schemes rely on a particular group structure to reduce math complexity

Agenda

- Class Overview
- Dependability
- Future Classes
- **Q&A**